

III

TYPOLOGY ANALYSIS: FAN - IN

What I will do in this analysis

Step 1 — Degree. I count how many different accounts send money to each receiving account, and keep only those receiving from 3 or more. This is the minimum degree that can be a fan-in.

Step 2 — Time window. I look at the gaps between incoming transactions to find the time scale of a "burst".

Step 3 — Selecting a threshold for the ML model. I use the accounts with the lowest time dispersion to measure how long their bursts last, and take that as my threshold.

Step 4 — Clustering. I run a DBSCAN model on each receiving account, using that threshold, so the model groups transactions that arrived close together in time. Each group is a candidate burst.

Step 5 — Analysing clusters. For every cluster I count how many different senders it has, how long it lasted, and how many transactions it holds. A cluster is a fan-in candidate if it has 3 or more distinct senders.

Step 6 — Conclusion. I look at what my definition selected, and compare it with how AMLSim defines fan-in in its Patterns file. The two criteria are not the same, and explaining why they differ is the main finding of this work.

This analysis uses the IBM Transactions for Anti-Money Laundering dataset (LI-Small variant), which contains 6,924,049 transactions between 712,688 bank accounts over a period of about 16 days.

The data is synthetic: it was produced by a simulator, not collected from a real bank

All the work here was done in Python, using Pandas for data handling and scikit-learn for the DBSCAN clustering model. Power BI was used for the initial profiling of the accounts table.

1

Degree and Definition**1.1. Introduction**

Fan-in is a money laundering typology where one account receives money from many different accounts. But there is more than one way to define it precisely, and the definition I choose shapes everything I detect.

The literature offers two main definitions. The structural one (used by AMLSim, the simulator that generated this dataset) defines fan-in purely by shape: one account connected to $k \geq 2$ different sending accounts, with no condition on time. The temporal one (used in detection work like MonLAD) adds speed: many sources sending into one account within a short time window, because launderers move money fast to reduce risk.

I choose the temporal definition. My reasoning is that a burst of many transactions from many accounts in a few minutes is more anomalous, and easier to separate from normal activity, than the same transactions spread across weeks. So throughout this work, "fan-in" means: an account receiving money from 3 or more different accounts, concentrated in a short time window.

I set the degree at 3 or more because receiving from 1 account cannot be fan-in by definition, and receiving from 2 is a weaker case I set aside (explained below). The time window is not fixed in advance: I derive it from the data itself in the sections that follow.

1.2. Why the time window matters in practice

For a bank or a fintech, the difference between the two definitions is not academic. A purely structural rule flags every account that receives from many sources, which includes shops, payroll accounts, and platforms collecting payments (a volume of alerts no compliance team can review). Adding a time condition targets something more specific: money that arrives from many sources and is moved fast.

The reason this could matter is that speed is part of the launderer's strategy, not an accident. Funds are split into small amounts across many transfers to stay under detection thresholds, and completed quickly because the sooner the money moves, the lower the risk of being caught. A detection rule built around short bursts is therefore aimed at the behaviour itself, not just at the shape of the network.

1.3. Obtaining a 3-Degree Subset

First I will check how many different accounts each "to_account" account is receiving transactions from:

```

distinctSendersPerAccount = dftransactions.groupby("to_account")["account"].nunique()

distinctSendersPerAccount

# here I see 576,176 results. A lot of them are related just to one or two accounts.

```

[157] ✓ 5.4s

to_account	# account
10042B660	785
10042B6A8	478
10042B6F0	176
10042B738	70
10042B780	118
10042B7C8	30
10042B810	34
10042B858	66
10042B8A0	81
10042B8E8	34

576,176 rows x 1 cols 10 per page << < Page 1 of 57618 > >>

There are 576,176 results. That is, $576,176 / 712,688$ different accounts receive transactions from other accounts.

I would like to see now how these receiving accounts are composed in relation to the amount of accounts they are receiving transactions from.

`distinctSendersPerAccount.value_counts()`

[165] ✓ 0.1s

# account	# count
1	223040
3	138043
2	121290
4	57517
5	19396
6	7011
7	2901
8	1638
9	960
10	631

93 rows x 1 cols 10 ▾ per page

So this means 223,040 different accounts are receiving transactions from one single account; 121,290 from 2 different accounts; 138,043 from 3; 631 from 10 and so on.

For my analysis, as I said in the introduction, I am going to filter the 344,330 / 576,176 accounts (almost 60%, leaving 231,846 different accounts) that receive transactions from just 1 or 2 accounts (223,040 + 121,290 in `distinctPerAccount.value_counts()` table).

Justifying leaving out 2 degree fan-in is more complex. In this case I see these accounts represent approximately 21% of receiving accounts. Money laundering with money coming from 2 different accounts to many different accounts could be possible, but it requires a lot of effort, making it less probable and, even if it is possible, I think it is more cost-effective to focus first on those accounts that receive money from more than 2 accounts, setting that subset aside for separate study, as it would need more crossed information to be analysed and get a meaningful conclusion.

I will focus then on the 40% left (231,846 different accounts).

In order to do that, I will have to filter from the original dataframe those transactions made to accounts that receive more than 2 transactions:

The screenshot shows a Data Wrangler interface. The code editor at the top contains the following Python code:

```
x_1 = distinctSendersPerAccount[distinctSendersPerAccount > 2]

focusGroup = dftransactions[dftransactions["to_account"].isin(x_1.index)]

focusGroup
```

Below the code editor, a status bar indicates a successful execution: [189] ✓ 4.8s Open 'focusGroup' in Data Wrangler.

The data table below shows the first 15 rows of the 'focusGroup' dataset. The columns are 'id', 'timestamp', and 'from_bank'.

	id	timestamp	from_bank
0	0	2022-09-01 00:08:00	11
4	4	2022-09-01 00:00:00	20
5	5	2022-09-01 00:24:00	12
6	6	2022-09-01 00:17:00	11
7	7	2022-09-01 00:07:00	11
8	8	2022-09-01 00:28:00	1120
9	9	2022-09-01 00:22:00	1217
10	10	2022-09-01 00:28:00	224
12	12	2022-09-01 00:29:00	3618
14	14	2022-09-01 00:04:00	1291

At the bottom, the interface shows '5,318,124 rows x 11 cols' and a pagination control set to '10' per page.

We are left with 5,318,124 / 6,924,049 (almost 77% of all transactions).

focusGroup then is the part of dftransactions with receiving accounts receiving transactions from 3 or more sending accounts.

2

Time Window: Timestamps and Time Gaps

2.1. Creating a Time Table

Let's see first what is the total time window of focusGroup:

```
focusGroup["timestamp"].describe()
```

[190] ✓ 0.5s

timestamp	
count	5318124
mean	2022-09-05 11:53:29.441790720
min	2022-09-01 00:00:00
25%	2022-09-02 10:19:00
50%	2022-09-05 18:28:00
75%	2022-09-08 05:32:00
max	2022-09-17 15:28:00

7 rows x 1 cols 10 per page

Approximately 16 days.

Until September 2nd: 25%

Until September 5th: 50%

Until September 8th: 75%

Until September 17th: 100%

They are more concentrated (75%) in the first half of the studied time window, where they are evenly distributed.

But what is most important here is not the time when transactions have been made, but rather **how time-concentrated transactions are with respect to a receiving account.**

And, to examine how time-concentrated transactions are, **I need to see the time gap between transactions sent to a receiving account.**

In order to see this, I am first going to create a two-column table in which the first column will show the receiving accounts and the second column will show the timestamp at which the transaction has been made and sort its values.

```
[196] ✓ 0.2s ⚙️ Open 'time_table_1' in Data Wrangler
```

...

	to_account	timestamp
0	8000ECA90	2022-09-01 00:08:00
4	8006AD530	2022-09-01 00:00:00
5	8006ADD30	2022-09-01 00:24:00
6	8006AD4E0	2022-09-01 00:17:00
7	8000ECA90	2022-09-01 00:07:00
8	81470DCF0	2022-09-01 00:28:00
9	8006AD4E0	2022-09-01 00:22:00
10	80567ED00	2022-09-01 00:28:00
12	800929EF0	2022-09-01 00:29:00
14	80092A730	2022-09-01 00:04:00

5,318,124 rows x 2 cols 10 per page

```
[197] ✓ 2.7s
```

```
time_table_1 = time_table_1.sort_values(["to_account", "timestamp"])
time_table_1
```

```
[48] ✓ 4.3s ⚙️ Open 'time_table_1' in Data Wrangler
```

...

	to_account	timestamp
157311	10042B660	2022-09-01 00:05:00
7112	10042B660	2022-09-01 00:08:00
72921	10042B660	2022-09-01 00:13:00
164374	10042B660	2022-09-01 00:13:00
151798	10042B660	2022-09-01 00:18:00
125130	10042B660	2022-09-01 00:24:00
448226	10042B660	2022-09-01 00:44:00
639387	10042B660	2022-09-01 04:45:00
634124	10042B660	2022-09-01 04:54:00
703412	10042B660	2022-09-01 06:02:00

5,318,124 rows x 2 cols 10 per page Page 1 of 531813

```
time_table_1 = time_table_1.sort_values(["to_account", "timestamp"])
time_table_1
```

I will also create a new column to translate timestamps to minutes, which might be useful later:

```
time_table_1["ts_min"] = time_table_1["timestamp"].astype("int64") / 60_000_000_000
time_table_1
```

[49] ✓ 0.0s Open 'time_table_1' in Data Wrangler

	to_account	timestamp	# ts_min
157311	10042B660	2022-09-01 00:05:00	27699.845
7112	10042B660	2022-09-01 00:08:00	27699.848
72921	10042B660	2022-09-01 00:13:00	27699.853
164374	10042B660	2022-09-01 00:13:00	27699.853
151798	10042B660	2022-09-01 00:18:00	27699.858
125130	10042B660	2022-09-01 00:24:00	27699.864
448226	10042B660	2022-09-01 00:44:00	27699.884
639387	10042B660	2022-09-01 04:45:00	27700.125
634124	10042B660	2022-09-01 04:54:00	27700.134
703412	10042B660	2022-09-01 06:02:00	27700.202

5,318,124 rows x 3 cols 10 per page << < Page 1 of 531813 > >>

2.2. Calculating the Gap Between Transactions

Now I am going to calculate the difference between each timestamp within the same account and translate that to minutes, because this will be useful for future operations:

```
time_table_1["gap"] = time_table_1.groupby("to_account")["timestamp"].diff()
time_table_1
```

[200] ✓ 1.6s Open 'time_table_1' in Data Wrangler

	to_account	timestamp	gap
157311	10042B660	2022-09-01 00:05:00	Missing value
7112	10042B660	2022-09-01 00:08:00	0 days 00:03:00
72921	10042B660	2022-09-01 00:13:00	0 days 00:05:00
164374	10042B660	2022-09-01 00:13:00	0 days 00:00:00
151798	10042B660	2022-09-01 00:18:00	0 days 00:05:00
125130	10042B660	2022-09-01 00:24:00	0 days 00:06:00
448226	10042B660	2022-09-01 00:44:00	0 days 00:20:00
639387	10042B660	2022-09-01 04:45:00	0 days 04:01:00
634124	10042B660	2022-09-01 04:54:00	0 days 00:09:00
703412	10042B660	2022-09-01 06:02:00	0 days 01:08:00

5,318,124 rows x 3 cols 10 per page << < Page 1 of 531813 > >>


```
time_table_1["gap"] = time_table_1["gap"].dt.total_seconds()/60
time_table_1
```

[201] ✓ 2.4s Open 'time_table_1' in Data Wrangler

	to_account	timestamp	# gap
157311	10042B660	2022-09-01 00:05:00	Missing value
7112	10042B660	2022-09-01 00:08:00	3.0
72921	10042B660	2022-09-01 00:13:00	5.0
164374	10042B660	2022-09-01 00:13:00	0.0
151798	10042B660	2022-09-01 00:18:00	5.0
125130	10042B660	2022-09-01 00:24:00	6.0
448226	10042B660	2022-09-01 00:44:00	20.0
639387	10042B660	2022-09-01 04:45:00	241.0
634124	10042B660	2022-09-01 04:54:00	9.0
703412	10042B660	2022-09-01 06:02:00	68.0

5,318,124 rows x 3 cols 10 per page << < Page 1 of 531813 > >>

```
time_table_1["gap"].describe()
```

[208] ✓ 0.2s

# gap	
count	5086278.0
mean	564.4090818472762
std	1057.991802229532
min	0.0
25%	6.0
50%	22.0
75%	854.0
max	22856.0

8 rows x 1 cols 10 per page

Analysing the gap table, we can see there is:

A mean of 564.41 minutes (9.4 hours)

A minimum value of 0.0: this means at least one transaction has been transferred in less than a minute.

Quartiles are the following:

25%: 6 minutes.

50%: 22 minutes.

75%: 854 minutes (around 14 hours).

A maximum value of 22,856 minutes (almost 16 days).

3

Finding an *epsilon* for temporal concentration

3.1. Three-Stage Approach

I started by thinking about which measure could tell me whether the incoming transactions to an account were concentrated in time.

Once I had created the attribute gap, I first considered building a table with one row per receiving account, containing dispersion measures of the gaps between incoming transactions: range, standard deviation and percentiles.

I discarded the percentile after thinking about what it actually measures. The percentile orders the gaps by size, not by time¹. So if an account received two transactions close together, then a long pause, then two more close together, and so on across the month, the lower percentiles would show short gaps, but there was no burst, just recurring activity. The percentile tells me how short the short gaps are, but not whether they happened one after another.

I also thought that standard deviation is sensitive to extreme values. But I concluded it is still useful in one of its extremes: if dispersion is low, I know the transactions were concentrated, without needing anything else.

The problem is with accounts where dispersion is high, because there I cannot tell whether there was a burst plus an isolated late transaction, or genuinely spread-out activity.

So I decided on a three-stage approach:

- 1) use dispersion to identify the clearly concentrated accounts;
- 2) try to identify a time-pattern in this subset;
- 3) use this information to build a clustering (DBSCAN) model to identify fan-in transactions.

¹ For instance, if we had gaps 6m, 1440m, 3m, 9m, then they would first be ordered by size like this: [3, 6, 9, 1440] and, p25 would be 3, p50 6 and so on. So p50 would definitely show gaps that are under 6 minutes, but that does not mean they happened in order.

Indeed, DBSCAN needs a parameter (eps) that defines how close two transactions must be to belong to the same episode or cluster.

So I decided to use the accounts with low dispersion: in those accounts the concentration is visible, so I can measure how long their bursts typically last and use that as eps.

3.2. Timestamp Dispersion for Each Receiving Account

First I am going to create a table with the standard deviation of every timestamp group (converted to minutes, which was the column `ts_min`) for every receiving account, to examine dispersion within each group.

The screenshot shows a Jupyter Notebook cell with the following code:

```
timestampDeviation = time_table_1.groupby("to_account")["ts_min"].std()
timestampDeviation
```

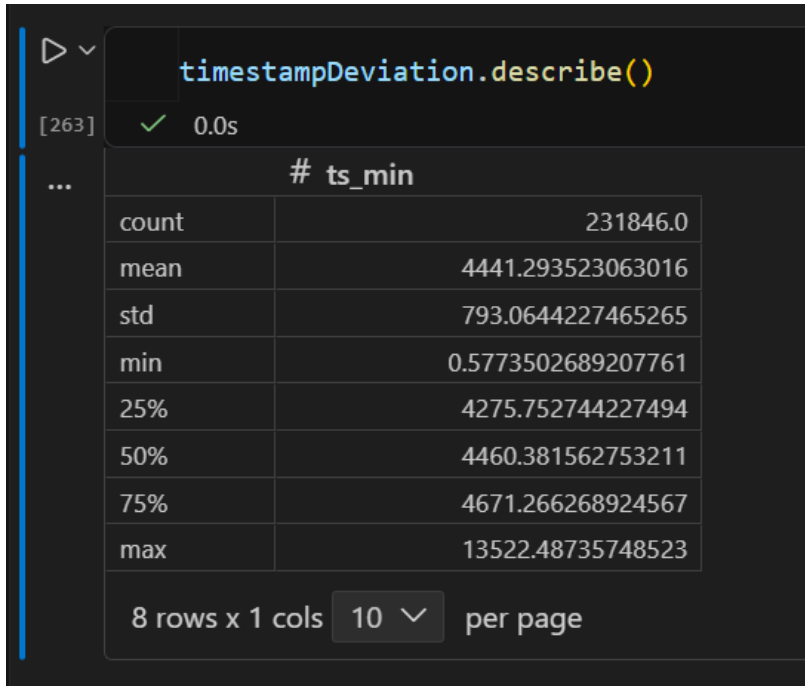
Below the code, a status bar indicates the execution was successful: [257] ✓ 1.2s Open 'timestampDeviation' in Data Wrangler.

The Data Wrangler interface displays a table with two columns: `to_account` and `# ts_min`. The table contains 10 rows of data, showing the standard deviation of timestamps for various receiving accounts.

to_account	# ts_min
10042B660	5081.979946890759
10042B6A8	5078.704813587103
10042B6F0	5113.733325801106
10042B738	5122.918741385623
10042B780	5216.620911341284
10042B7C8	5340.792634865475
10042B810	5103.098783206103
10042B858	5091.743817677716
10042B8A0	5067.891726814774
10042B8E8	5127.181171158744

At the bottom of the table, it indicates "231,846 rows x 1 cols" and a dropdown menu set to "10" per page.

Now I want to see its basic analysis:



```
timestampDeviation.describe()
```

[263] ✓ 0.0s

	# ts_min
count	231846.0
mean	4441.293523063016
std	793.0644227465265
min	0.5773502689207761
25%	4275.752744227494
50%	4460.381562753211
75%	4671.266268924567
max	13522.48735748523

8 rows x 1 cols 10 per page

So we have:

A mean of $\approx 4,441$ min. (≈ 74 hours ≈ 3 days).

A minimum value of ≈ 0.6 min.

Quartiles:

25 % $\approx 4,276$ min. ≈ 71 hours ≈ 3 days.

50 % $\approx 4,460$ min. ≈ 74 hours ≈ 3 days.

75 % $\approx 4,671$ min. ≈ 78 hours ≈ 3.25 days.

A maximum value of $\approx 13,522$ min. ≈ 225 hours ≈ 9.4 days.

This is showing a really low minimum value and very high maximum value, but most values are closer to the mean value (75 % of values are under ≈ 4672 min.) and quartile values are not far from each other.

So this is not giving me the information I was looking for. I am going to look at what is going on at lower percentiles:

```
timestampDeviation.quantile([0.01, 0.02, 0.03, 0.04, 0.05, 0.10, 0.15, 0.20])
```

281] ✓ 0.0s

...

	# ts_min
0.01	553.7006716173267
0.02	1350.4433221198283
0.03	2301.8828794894534
0.04	3121.694034319258
0.05	3755.335132176651
0.1	4103.881943675879
0.15	4178.761848971618
0.2	4231.62645267168

8 rows x 1 cols 10 per page << < Page 1 of 1 > >> 🔍 📄

Even in these low percentiles we still see that a value as low as 1% shows 553 minutes (≈ 9.2 hours). So I will isolate this 1% to look at it closer.

```
corte = timestampDeviation.quantile(0.01)
timestampDeviation_percentile1 = timestampDeviation[timestampDeviation <= corte]
timestampDeviation_percentile1.describe()
```

[308] ✓ 0.0s

...

	# ts_min
count	2319.0
mean	210.4833736682926
std	189.4152562002982
min	0.5773502689207761
25%	9.729596134078086
50%	186.93606101854013
75%	385.4804348186419
max	553.3922659380053

8 rows x 1 cols 10 per page

So here we have:

A mean of ≈ 210 min. (≈ 3.5 hours).

A minimum value of ≈ 0.6 min.

Quartiles:

25 % ≈ 9.7 min.

50 % ≈ 187 min. ≈ 3 hours

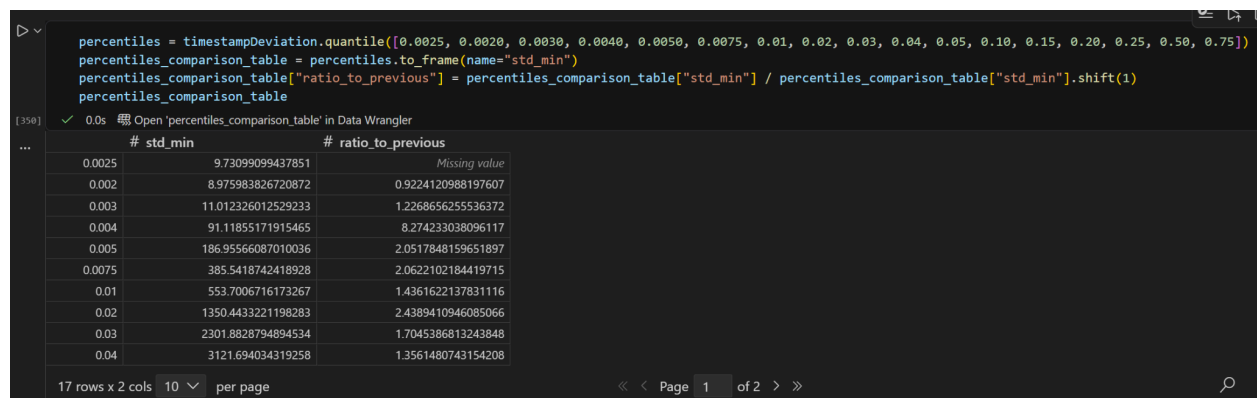
75 % ≈ 385 min. ≈ 6.4 hours

A maximum value of ≈ 553 min. ≈ 9 hours

So we see there is a relevant difference between percentiles 25 and 50, being $9.7 \approx 0.05$ of 187.

So the threshold I am looking for might be between percentiles 50 and 25.

Let's see why this stands comparing quartiles and percentiles in terms of magnitude between them:



The screenshot shows a Data Wrangler interface with a table of percentiles. The table has three columns: '# std_min' and '# ratio_to_previous'. The first column contains values for percentiles from 0.0025 to 0.04. The second column contains the corresponding ratio to the previous percentile. The table shows a significant jump in the ratio between percentiles 0.0040 and 0.0030.

# std_min	# ratio_to_previous
0.0025	Missing value
0.002	0.9224120988197607
0.003	1.2268656255536372
0.004	8.274233038096117
0.005	2.0517848159651897
0.0075	2.0622102184419715
0.01	1.4361622137831116
0.02	2.4389410946085066
0.03	1.7045386813243848
0.04	1.3561480743154208

Here I compare the ratio each percentile has with respect to the next smaller percentile and we can see there is an eight-fold jump between percentiles 0.4 (0.0040) and 0.3 (0.0030), which is between 8 and 4 times higher than the other visible jumps.

I am going to look then at what is happening inside this percentile 0.4 (0.004):

```
percentileCut = timestampDeviation.quantile(0.004)
thresholdSet = timestampDeviation[timestampDeviation <= percentileCut]
thresholdSet.describe()
```

[410] ✓ 0.0s

... # ts_min

count	928.0
mean	15.86122409041807
std	19.35157274430428
min	0.5773502689207761
25%	7.6381860318143175
50%	8.975876524825875
75%	11.010600639957836
max	91.04211479716027

8 rows x 1 cols 10 per page

We can see there is a maximum value much higher than the value of percentile 75.

I will now filter from our original time table called `time_table_1` these accounts that are part of percentile 0.4 of the timestamps standard deviation. That is, the 0.4% most time-concentrated receiving accounts.

3.3. Choosing a Candidate

So let's see what is going on in terms of timestamps and gaps here:

```
a = thresholdSet.index

focusGroup_2= time_table_1[time_table_1["to_account"].isin(a)]
focusGroup_2
```

[416] ✓ 1.3s Open 'focusGroup_2' in Data Wrangler

... to_account timestamp # gap # ts_min

1455667	8000FE640	2022-09-01 22:34:00	Missing value	27701194.0
1455664	8000FE640	2022-09-01 22:37:00	3.0	27701197.0
1455663	8000FE640	2022-09-01 22:38:00	1.0	27701198.0
1455665	8000FE640	2022-09-01 22:51:00	13.0	27701211.0
1455666	8000FE640	2022-09-01 22:55:00	4.0	27701215.0
3708956	800125970	2022-09-05 21:30:00	Missing value	27706890.0
3708954	800125970	2022-09-05 21:31:00	1.0	27706891.0
3708950	800125970	2022-09-05 21:32:00	1.0	27706892.0
3708953	800125970	2022-09-05 21:32:00	0.0	27706892.0
3708960	800125970	2022-09-05 21:33:00	1.0	27706893.0

10,145 rows x 4 cols 10 per page << < Page 1 of 1015 > >>

Now I should see how these gaps are composed:

[434] `focusGroup_2["gap"].describe()`

...

# gap	
count	9217.0
mean	4.877508950851687
std	25.92582716674804
min	0.0
25%	0.0
50%	1.0
75%	3.0
max	706.0

8 rows x 1 cols 10 ▾ per page

SQLTools

[476] `focusGroup_2["gap"].quantile([0.99,0.98,0.95,0.9,0.8,0.7])`

...

# gap	
0.99	114.840000000000015
0.98	21.0
0.95	11.0
0.9	6.0
0.8	4.0
0.7	2.0

6 rows x 1 cols 10 ▾ per page

We can see 90% of the gaps of focusGroup_2 are under six minutes, before it jumps to almost its double (11).

Six minutes is then a good candidate to represent the concentration I was looking for as a first threshold to set my *epsilon* value for the DBSCAN model.

4

Creating a DBSCAN model

4.1. First Trial

Now I am going to create a DBSCAN model, being our *epsilon* the found value of 6.

I will first try it in a single row:

```
101] from sklearn.cluster import DBSCAN
```

✓ 0.0s

I am going to try first on a single row to test it

```
102] singleAccount = focusGroup_2["to_account"].iloc[0] #this is taking the first value of to_account
data_1 = focusGroup_2[focusGroup_2["to_account"] == singleAccount]
```

data_1

✓ 0.0s Open 'data_1' in Data Wrangler

	to_account	timestamp	# ts_min	# gap
1455667	8000FE640	2022-09-01 22:34:00	27701.194	Missing value
1455664	8000FE640	2022-09-01 22:37:00	27701.197	3.0
1455663	8000FE640	2022-09-01 22:38:00	27701.198	1.0
1455665	8000FE640	2022-09-01 22:51:00	27701.211	13.0
1455666	8000FE640	2022-09-01 22:55:00	27701.215	4.0

5 rows x 4 cols 10 per page

```

reshaped = data_1["ts_min"].values.reshape(-1, 1)
reshaped

```

[108] ✓ 0.0s Open 'reshaped' in Data Wrangler

```

... array([[27701.194],
          [27701.197],
          [27701.198],
          [27701.211],
          [27701.215]])

```

```

model = DBSCAN(eps=6, min_samples=3)
beta_1 = model.fit_predict(reshaped)

print(beta_1)

```

[118] ✓ 0.1s Open 'beta_1' in Data Wrangler

```

... [0 0 0 0 0]

```

It worked with the first row, as the result is [0,0,0,0], which means the model grouped these five transactions inside the same cluster.

4.2. Creating a function

So I will now create a function that will be applied to every group:

```

def cluster(group):
    X = group["ts_min"].values.reshape(-1, 1)
    return pd.Series(
        DBSCAN(eps=6, min_samples=3).fit_predict(X),
        index=group.index
    )

result = time_table_1.groupby("to_account", group_keys=False).apply(cluster)

```

[125] 14.9s

How does the function cluster work?

It uses an argument called group, which will be every to_account group, i.e., all the transactions made to each receiving account.

Inside cluster there is a new variable called X (not quite a creative name) that will be used to reshape every group from a 1D array to a 2D array (a matrix of n rows and one column); this is needed because DBSCAN works with +1 dimensional arrays and every group I am working with is naturally a 1D array, so it needs to be reshaped.

The function returns a Pandas series, built through a Pandas function called Series (pd.Series()) that needs basically two arguments: the series values and the index labels that will be correlated to each value. So here we are building values, which will be each array of numbers -which cluster each transaction belongs to-, associated to an index that corresponds to its position within the group.

Finally, the variable result is applying the function cluster to each group of transactions made to a single to_account account.

4.3. Is eps = 6 working?

```

result.value_counts()

```

[204] ✓ 0.1s

	#	count
0		5250245
-1		42160
1		25719

Here we can see that 98% of transactions are inside the first cluster of their group.

This means almost every transaction is inside a cluster and, the way I see it, not every transaction of a receiving account should be inside a cluster, because they were built precisely to detect anomalous concentrated transactions.

Having 98% inside the first cluster of every group is definitely not good for what I was looking for because money laundering should be anomalous even in the time axis.

This is probably explained by the chaining effect that can be seen in the DBSCAN model and probably 6 minutes is generating a big cluster per group.

4.4. Choosing 2 minutes as a more accurate *epsilon* value

The result above told me $\text{eps} = 6$ was too permissive: the chaining effect linked almost every incoming transaction of an account into a single cluster, which defeats the purpose of isolating bursts. So I went back to the percentile table that produced the original candidate:

`focusGroup_2["gap"].quantile([0.99,0.98,0.95,0.9,0.8,0.7])`

[476]

...

	# gap
0.99	114.84000000000015
0.98	21.0
0.95	11.0
0.9	6.0
0.8	4.0
0.7	2.0

6 rows x 1 cols 10 per page

Six minutes came from percentile 90 of the gaps in the most concentrated accounts (`focusGroup_2`). It accepts almost every gap in a subset that is already, by construction, the most time-concentrated in the dataset. Looking at the same distribution, percentile 70 sits at 2 minutes.

I chose 2 minutes because it comes from the same empirical source as the original candidate, so it is not an arbitrary number: it is a stricter cut of the same distribution. At this scale a burst means transactions arriving essentially really close to each other, which is more similar to the behaviour I wanted to detect in section 1.

I will then try reducing six to two minutes, because if we look back at the percentiles we had calculated before, percentile 0.7 of percentile 0.004 had 2 minutes:

```
def cluster(group):
    X = group["ts_min"].values.reshape(-1, 1)
    return pd.Series(
        DBSCAN(eps=2, min_samples=3).fit_predict(X),
        index=group.index
    )

result = time_table_1.groupby("to_account", group_keys=False).apply(cluster)
```

[215] ✓ 9m 7.4s

```
result.value_counts()
```

[216] ✓ 0.0s

	# count
0	2925316
1	1840556
2	384556
-1	134694
3	32117
4	885

6 rows x 1 cols 10 ▾ per page << < Page 1 of 1 > >>

With `eps = 2` the model produced five clustering groups and left 134,694 transactions outside any cluster, which is much closer to what I was looking for: clusters that isolate concentrated episodes rather than absorbing all of an account's activity.

5 Cluster Analysis

Now I should add the account attribute to my `time_table_1` table, so I can count the amount of different sending accounts per cluster:

time_table_1["from_account"] = focusGroup["account"]

time_table_1

[147] ✓ 0.8s Open 'time_table_1' in Data Wrangler

	to_account	timestamp	# ts_min	# gap	# cluster	from_bank	from_account
157311	10042B660	2022-09-01 00:05:00		27699.845	Missing value	0 81A731C90	81A731C90
7112	10042B660	2022-09-01 00:08:00		27699.848	3.0	0 800ABAD90	800ABAD90
72921	10042B660	2022-09-01 00:13:00		27699.853	5.0	0 80B676EE0	80B676EE0
164374	10042B660	2022-09-01 00:13:00		27699.853	0.0	0 81BD7B530	81BD7B530
151798	10042B660	2022-09-01 00:18:00		27699.858	5.0	0 819A0A4F0	819A0A4F0
125130	10042B660	2022-09-01 00:24:00		27699.864	6.0	0 81518B390	81518B390
448226	10042B660	2022-09-01 00:44:00		27699.884	20.0	0 801122660	801122660
639387	10042B660	2022-09-01 04:45:00		27700.125	241.0	0 81ASDA410	81ASDA410
634124	10042B660	2022-09-01 04:54:00		27700.134	9.0	0 807D47110	807D47110
703412	10042B660	2022-09-01 06:02:00		27700.202	68.0	0 80BB13AB0	80BB13AB0

5,318,124 rows x 7 cols 10 per page < < Page 1 of 531813 > >

I will now create a new table called cluster_summary in which I will group by receiving account and cluster to analyse every cluster inside every receiving account. This new table will contain how many different senders there are per cluster, what is the cluster's time window and how many transactions have been made by those senders.

After that I will filter the amount of senders to more than two, since we are trying to identify degree 3 fan-in suspicious accounts.

```
cluster_summary = (time_table_1[time_table_1["cluster"] != -1]
                    .groupby(["to_account", "cluster"])
                    .agg(senders=("from_account", "nunique"),
                        duration_min=("ts_min", lambda s: s.max() - s.min()),
                        amount_of_transactions_per_cluster = ("from_account", "count")))

filtered_cluster_summary = cluster_summary[cluster_summary["senders"] > 2]
```

[147] ✓ 44.0s

filtered_cluster_summary

[148] ✓ 0.1s Open 'filtered_cluster_summary' in Data Wrangler

	(to_account, 'cluster')	# senders	# duration_min	# amount_of_transactions...
...	('10042B660', np.int64(0))	785	2.87399999999997963	786
	('10042B660', np.int64(1))	767	1.43699999999980791	767
	('10042B6A8', np.int64(0))	478	2.5370000000000262	478
	('10042B6A8', np.int64(1))	473	1.43699999999980791	473
	('10042B6F0', np.int64(0))	176	2.84300000000007567	176
	('10042B6F0', np.int64(1))	169	1.43100000000004948	169
	('10042B738', np.int64(0))	70	2.84299999999971187	70
	('10042B738', np.int64(1))	65	1.41300000000004657	65
	('10042B780', np.int64(0))	118	2.84700000000015716	118
	('10042B780', np.int64(1))	105	1.41300000000004657	105

241,152 rows x 3 cols 10 per page < < Page 1 of 24116 > >

There are 241,152 clusters.

If we describe this table:

filtered_cluster_summary.describe()

[149] ✓ 0.0s

...

	# senders	# duration_min	# amount_of_transactions...
count	241152.0	241152.0	241152.0
mean	3.75354133492569	4.743153206276424	13.976587380573248
std	4.142619603306804	4.055490592911314	11.04957740223331
min	3.0	0.0	3.0
25%	3.0	2.00800000000163	7.0
50%	3.0	2.7889999999970314	10.0
75%	4.0	6.494999999998981	17.0
max	785.0	15.41700000000128	786.0

8 rows x 3 cols 10 ▾ per page << < Page 1 of 1 > >>

We can see there are from 3 to 785 senders, (which is precisely the criterion I had chosen when I chose to analyse accounts receiving transactions from three or more different accounts) but p75 is 4, so most accounts come from 3 or 4 different accounts, with outliers in p100 - p75.

The duration in minutes goes from 0 to 15 minutes, which is precisely what I wanted to identify with the DBSCAN model: burst of several transactions in a few minutes. Most clusters have 6 or less minutes of duration, with a mean of ~5 minutes.

The amount of transactions have a mean of ~14, with p75 being 17.

This means most suspected accounts are receiving between 3 and 17 transactions in between 0 and 7 minutes from between 3 and 4 senders.

6 Conclusion

This work defined fan-in as a burst: many different accounts sending money into one account **within a short time window**. Using this definition, I identified 211,099 receiving accounts out of the 231,846 that had 3 or more distinct senders — that is, 91%. The time criterion filtered out less than 10% of the candidates. In this dataset, receiving from several accounts within a short window is not rare: it is the normal case.

In this dataset, short bursts turned out to be common rather than exceptional, so the time window alone did not reduce the candidate set much. In a real institution the same rule would need to be combined with other signals — transaction amounts, customer profile, or the account's usual behaviour — before it could be used to raise alerts. This part of the work though has been focused on fan-in behaviour, not on every other signal worth looking at.

The dataset comes with a Patterns file listing the laundering structures the simulator planted. It contains 12 fan-in blocks. That number is very far from mine, but the two are not measuring the same thing, and it is worth explaining why.

AMLSim defines fan-in structurally: one account connected to $k \geq 2$ sending accounts, with no condition on time. Its blocks confirm this: the transactions in each one are spread across several days, not concentrated in minutes. My definition adds a temporal condition that theirs does not have. So the two criteria select different sets by construction, and neither one is the correct version of the other.

There is a second and more important difference worth mentioning.

In AMLSim, fan-in exists both as a normal pattern and as a laundering pattern, with the same shape. A shop receiving payments from many customers looks structurally identical to a laundering fan-in. The 12 blocks are the ones the simulator deliberately planted and labelled; the rest of the fan-in shapes in the data are ordinary activity. The difference between them is not in the transactions but in a label assigned when the data was generated.

This is what makes the structural approach reach its limit. My model identifies the fan-in shape combined with a temporal dimension, and it does so correctly. What no shape-based rule can do (neither mine nor a purely structural one) is separate laundering fan-ins from legitimate ones, because they share the same form. Doing that would require information beyond topology and timing: amounts, currencies, or tracing where the funds came from. This is the core difficulty of typology-based detection in real anti-money-laundering work.